MLPC Report

Team Quantized Encoders

Samuel Eder **Andreas Resch**

1 Dataset Preparation

1.1 Label Aggregation

The annotations are transformed into labels of shape [T, C] by computing the mean over the annotator dimension. Each value is now a fraction representing how many annotators marked a class active in a given time segment. Then these soft labels are converted to hard labels using a threshold. Values >= 0.5 are converted to 1, everything else becomes 0.

For classical models and metric computation, these soft labels are binarized at 0.5. For CNN training, however, the soft label values are used directly with binary cross-entropy. During evaluation, ground-truth labels are still binarized at 0.5 so that F1 scores remain comparable across all models.

For multi-label sound event detection systems, this is a suitable strategy because averaging across annotators preserves information and the exploration phase showed that class labeling can differ across annotators, especially for short sound signals. The method is simple and reproducible. Averaging also reduces the impact of single annotator mistakes. One limitation is that all annotators are weighted equally and the hard threshold can remove weak or disputed events, for example short events marked by only one annotator.

1.2 Data Split

We created two variants of the data split: a recording-grouped split and a collector-grouped split. In both cases, splitting was performed at the recording level rather than at the frame level. First, the recordings were divided into a development set containing 80% of the data and a hold-out test set containing the remaining 20%.

The recording-grouped split prevents leakage between overlapping segments originating from the same recording. The collector-grouped split is stricter: all recordings from the same collector are assigned to the same subset. This design was motivated by the assumption that audio files from the same collector might share a unique acoustic fingerprint, which could otherwise leak into both training and evaluation data and artificially inflate performance.

In later experiments, we observed no significant difference between the recording-grouped and collector-grouped splits. Therefore, the collector-grouped split was not used in further experiments.

We checked the resulting class distributions by counting the fraction of active frames per class in each split. Similar class frequencies are important to ensure fair evaluation, especially for less frequent classes. Because we split by recording or collector, we avoid information leakage but cannot ensure perfectly balanced class distributions every time. We prioritized leakage avoidance, but still checked that no big imbalances are in the splits. The distributions can be seen in Figure 1.

1.3 Preprocessing

For preprocessing we first converted invalid values to missing ones. Then we used mean imputation and standardized all features to have a mean of zero and variance of one. Imputation and standardization were only fitted on the training data within each fold. This process is important because of the different numerical scales for different audio features and classifiers might be sensitive to feature magnitudes. We did not apply explicit feature selection in this preparation step, but kept all 960 feature dimensions and left feature reduction to later model-selection experiments.

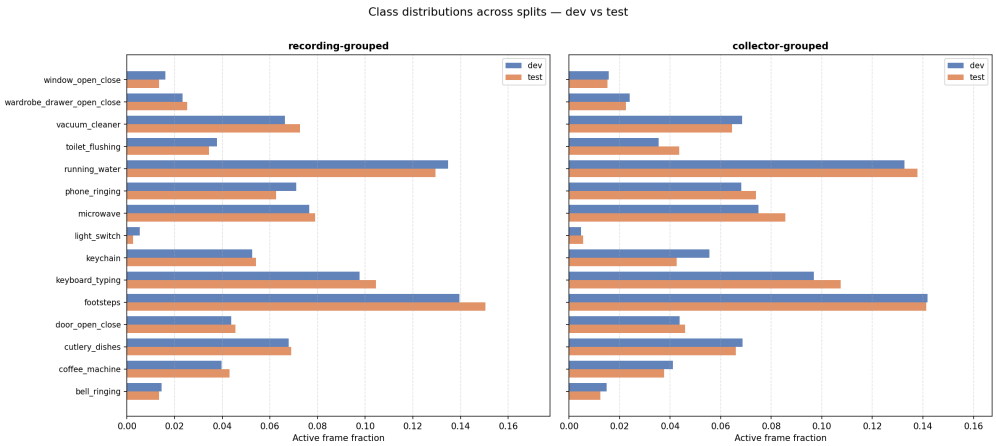

Figure 1: Class Distributions across recording-grouped split and collector-grouped split

2 Evaluation

2.1 Evaluation Criterion

Since multiple sound classes per frame can be active at once and since some of the classes are short and rare or ambiguous, we chose F1 as an evaluation metric for this multi-label-classification task. F1 includes precision (when the model predicts a class, how often is it correct) and recall (when a class is actually present, how often does the model find it) and therefore is better suited, since it ignores the huge number of true negatives, which is what you want in sparse audio-event data. Macro F1 calculates F1 per class and then averages all classes equally. That means that events with a shorter time span, such as `light_switch` or `window_open_close` matter just as much as events that are continuously active over multiple frames such as `running_water`. Micro F1 as a second metric aggregates true positives, false positives, and false negatives across all classes before computing precision, recall, and F1. Frequent classes contribute more strongly to the final score, therefore answering a slightly different question: "How good is the model overall across all event decisions?"

2.2 Baseline Performance

The empirical class-frequency sampling baseline establishes a no-feature lower bound. The results are shown in Table 1. This baseline is useful as a performance floor, not as a definition of good performance. The empirical class-frequency shows what can be achieved from class priors without using any audio features only. The theoretical best possible performance would be F1 = 1.0 for all 15 classes, but in practice perfect performance is unlikely, because the data contains sparse, short, overlapping, and sometimes ambiguous audio events with annotator disagreement and uncertain temporal boundaries.

Split strategy	**Macro F1**	**Micro F1**
Recording-grouped CV	0.0589 ± 0.0004	0.0853 ± 0.0010
Collector-grouped CV	0.0589 ± 0.0007	0.0853 ± 0.0011

Table 1: Empirical class-frequency sampling baseline performance.

3 Experiments

3.1 Classifiers and Hyperparameters

For this task we chose Logistic Regression, Random Forests and a CNN.

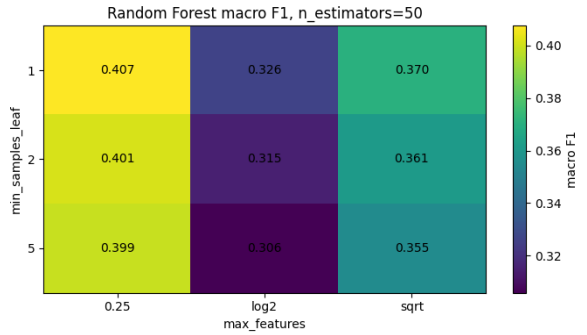

Figure 2: Heatmap Visualization of Macro F1 for different Hyperparameter settings for Random Forest Classifier

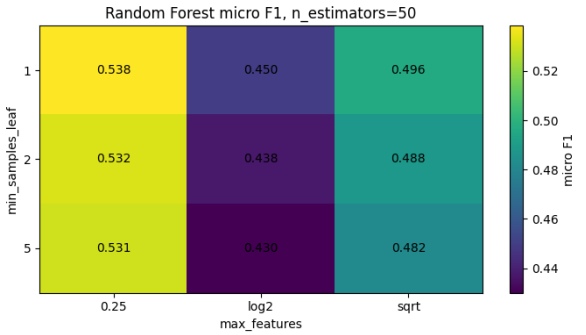

Figure 3: Heatmap Visualization of Micro F1 for different Hyperparameter settings for Random Forest Classifier

3.1.1 Logistic Regression and Random Forests

The models were evaluated with recording-grouped 5-fold cross-validation on the development split, using macro F1 as the main selection criterion because the dataset is multi-label and strongly class-imbalanced.

For Logistic Regression, the most relevant hyperparameter is the inverse regularization strength C. Smaller values of C enforce stronger regularization and therefore simpler linear decision boundaries, while larger values allow the model to fit the training data more flexibly. We tested $C \in \{0.01, 0.1, 1.0, 10.0\}$. Since $C = 10.0$ achieved the highest recording-grouped CV macro F1 (0.4624 ± 0.0071), it was selected as the final Logistic Regression setting.

C	Macro F1 (CV)	Micro F1 (CV)
0.01	0.4393 ± 0.0066	0.5612 ± 0.0028
0.1	0.4587 ± 0.0073	0.5724 ± 0.0021
1.0	0.4622 ± 0.0069	0.5731 ± 0.0029
10.0	0.4624 ± 0.0071	0.5729 ± 0.0029

Table 2: Logistic Regression recording-grouped CV performance for different regularization strengths C.

For the Random Forest, we ran single test runs with fixed $max\_features = "sqrt"$ and $min\_samples\_leaf = 1$ and varied $n\_estimators \in 50, 100$. The results showed that for this configuration $n\_estimators = 50$ performed better and finished faster, therefore we fixed $n\_estimators = 50$ for the grid search. $max\_features$ is the number of features considered at each split, controlling randomness and diversity of trees. $min\_samples\_leaf$ is the minimum number of samples required in a leaf node, controlling how smooth the trees become. The grid we searched evaluated each combination of $max\_features \in \{"sqrt", "log2", 0.25\}$ and $min\_samples\_leaf \in \{1, 2, 5\}$. The best Random Forest configuration was $max\_features = 0.25$ and $min\_samples\_leaf = 1$, reaching 0.4074 ± 0.0026 macro F1 and 0.5384 ± 0.0051 micro F1. The full results of the grid search are visualized in Figures 2 and 3.

Overall, Logistic Regression achieved the stronger validation performance, suggesting that the standardized acoustic feature representation is well suited for a regularized linear classifier, while the Random Forest was less effective on this high-dimensional feature space.

3.1.2 Convolutional Neural Network

As a second non-linear model family, we trained a raw mel-spectrogram CNN. The final CNN setup uses 1.0 s mel-spectrogram patches with 64 mel bands and 0.3 s pre-context before each frame. The model uses independent sigmoid outputs for the 15 classes because the task is multi-label. Unlike the classical models, CNN is trained directly on soft labels with binary cross-entropy, while evaluation targets are binarized at 0.5 for comparable F1 scores.

CNN hyperparameters were selected inside the development data only. We used a small grid over learning rate, dropout, and batch size. Learning rate candidates covered conservative to aggressive Adam updates; dropout controlled regularization; and batch size was tested because larger batches can use the MPS GPU more efficiently but also change the optimization dynamics. To avoid leakage, we used an inner recording-grouped validation fold within the development data. The held-out test set was not used for hyperparameter selection.

The selected final CNN setting used learning rate 0.003, dropout 0.1, batch size 256, weight decay 0.0, and 10 final training epochs.

3.2 Performance Comparison

Table 3 reports the final held-out test performance using the decision threshold of 0.5. CNN performs best overall, with macro F1 = 0.5021 and micro F1 = 0.6450. Logistic Regression is the strongest classical baseline, reaching macro F1 = 0.4637 and micro F1 = 0.5843. Random Forest performs lower, with macro F1 = 0.4050 and micro F1 = 0.5426. This indicates that temporal-spectral context from raw mel-spectrogram patches improves both balanced per-class performance and overall frame-level decisions. All three chosen models outperform the empirical class-frequency sampling baseline as shown in Figure 4.

Model	Macro F1	Micro F1
CNN	0.5021	0.6450
Logistic Regression	0.4637	0.5843
Random Forest	0.4050	0.5426

Table 3: Final held-out test performance using the decision threshold of 0.5.

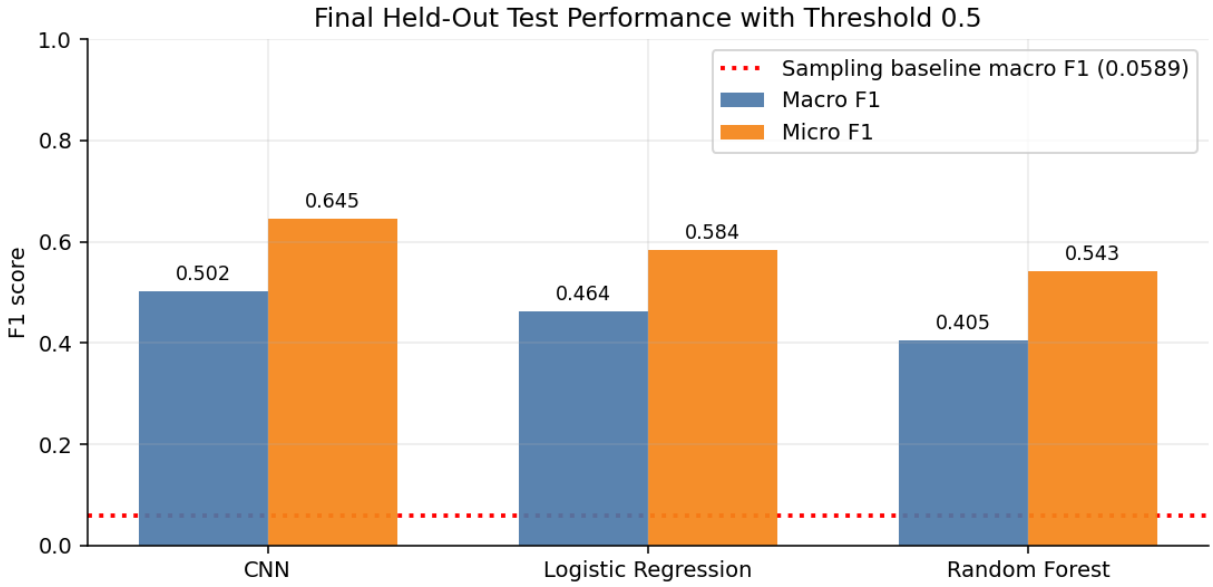

Figure 4: Final held-out test performance using the decision threshold of 0.5.

4 Case Study and Reflection

4.1 Case Study

For the case study we used two recordings from the held out test set and the predictions given by Logistic Regression. We chose Logistic Regression for this because it delivers middle-ground results which are easy to interpret. In Figure 5 we have an audio file that contains longer target sounds that are recognized reasonably well. In Figure 6 we see short target sounds that are difficult to recognize. Also interesting is that in Figure 5 the footsteps are predicted well and in Figure 6 the footsteps are not predicted at all. In Figure 5, also `cutlery_dishes` gets predicted correct approximately 50%. `running_water` the prediction is only slightly longer than the expectation and also `microwave` is predicted very well. In Figure 6 we see that nearly nothing is predicted and the only prediction doesn't match the expectation.

Overall, the results from the case study align with the results from Figure 7. Short target sounds like `light_switch` and `window_open_close` are difficult for our Logistic Regression, longer target sounds like `running_water` or `microwave` are handled better.

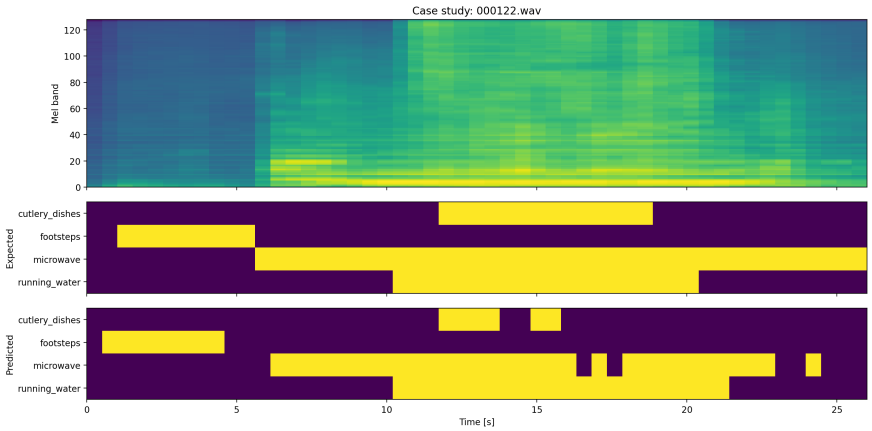

Figure 5: Mel Spectogram (top) and True labels (middle) vs. expected labels (bottom) for audio file 000122.wav

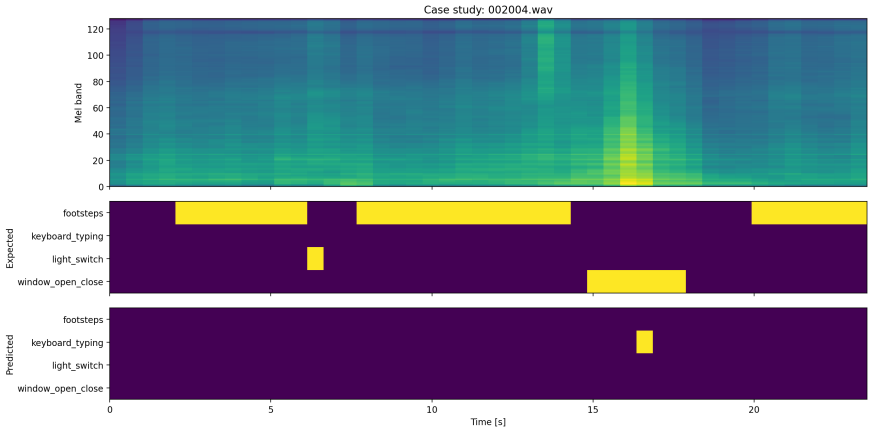

Figure 6: Mel Spectogram (top) and True labels (middle) vs. expected labels (bottom) for audio file 002004.wav

4.2 Reflection

Overall, CNN achieves the best results with macro F1 = 0.5021 and micro F1 = 0.6450, see Table 3. As shown in Figure 7, the per-class analysis reveals a clear pattern: sound events that are long-duration and acoustically distinct, such as `running_water` (F1 = 0.83), `vacuum_cleaner` (F1 = 0.71) and `phone_ringing` (F1 = 0.71), are detected reliably. In contrast, short transient events such as `light_switch` (F1 = 0.00) and `window_open_close` (F1 = 0.05) remain nearly undetectable.

This performance gap is related to the frame-level evaluation setup. A brief click or switch event activates only a small number of frames, and the fixed 0.5 threshold may not be sensitive enough to capture these sparse activations. Class-specific threshold tuning would address this by lowering the decision threshold for rare and short-duration classes, trading some precision for better recall on underrepresented events. The aggregation of time context in the dataset, averaging over annotators and frames, further blurs the signal for brief events, making them hard to distinguish from background. A further direction for improvement is more extensive hyperparameter search: the current grid was kept small to avoid overfitting the development fold, so better local minima likely remain unexplored.

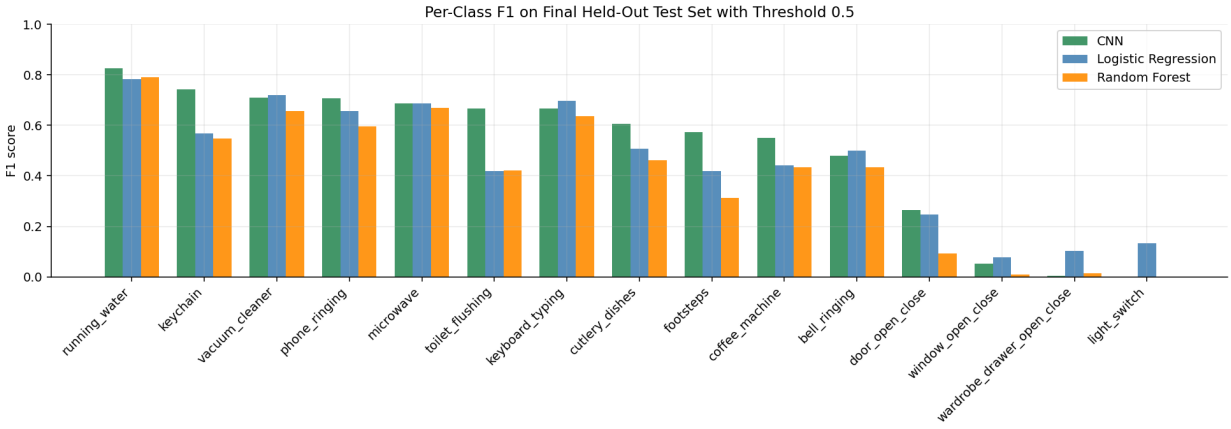

Figure 7: Per-class F1 on the final held-out test set using the decision threshold of 0.5.

5 Disclosure of LM and AI Tool Use

Throughout the creation of this report, Large Language Models (LLMs) were used to support various stages of the workflow, especially coding and improving the clarity and quality of our writing.

For coding parts we used Claude Code as well as ChatGPT Thinking. For writing improvements mainly ChatGPT Thinking was used.

All generated code suggestions and textual suggestions were checked manually. Code was verified by running the corresponding notebooks, inspecting the produced outputs, and comparing the reported numbers with the saved experiment results. Textual suggestions were only included if they matched our own experiments and interpretations.